WAN 高质量超分方案整理

本文记录当前在 LightX2V 中探索的 WAN 高质量超分主线：在 WAN `changing_resolution` 的切分辨率步骤中，保留原 diffusion 调度闭环，但把原本的 latent 插值替换为借助 LTX2 upsampler 权重的无训练桥接方案。

1. 目标

原始 WAN `changing_resolution` 的核心做法是：

```
x_t -> x0_pred -> trilinear interpolate -> add_noise -> continue diffusion
```

它可以低成本地把低分 latent 切到高分 latent，但插值本身没有学习到视频细节恢复能力。当前目标是：

- 保留 WAN 采样流程中的 `x_t -> x0_pred -> add_noise -> continue diffusion` 逻辑。
- 只替换“把近似干净样本 `x0_pred` 迁移到下一阶段分辨率”这一段。
- 不训练任何新模型。
- 复用 LTX2 已有的 spatial x2 latent upsampler 权重。
- 先把分辨率策略收窄为单次 `0.5 -> 1.0`，避免倍率不匹配。

2. 当前实现概览

新增链路没有直接修改原有 `changing_resolution` scheduler，而是新增了独立入口：

- 新 runner：`lightx2v/models/runners/wan/wan_ltx2_bridge_runner.py`
- 新 scheduler：`lightx2v/models/schedulers/wan/ltx2_bridge_resolution/scheduler.py`
- 新 bridge：`lightx2v/models/video_encoders/hf/wan/upsampler/ltx2_pixel_bridge.py`
- 新配置：`configs/changing_resolution/wan_t2v_ltx2_bridge.json`
- 新脚本：`scripts/changing_resolution/run_wan_t2v_ltx2_bridge.sh`

新模型入口注册为：

```
--model_cls wan2.1_ltx2_bridge
```

当前只支持：

- `wan2.1`
- `t2v`
- 单次空间 x2 升分
- `resolution_rate=[0.5]`
- 不支持 `lazy_load`
- 不支持 `unload_modules`
- 不支持 `use_tae`

3. 新链路流程

新链路只发生在 `changing_resolution_steps` 命中的切换步。完整流程是：

```
x_t
-> x0_pred
-> WAN decode
-> LTX2 encode
-> LTX2 upsample
-> LTX2 decode
-> WAN encode
-> add_noise
-> continue diffusion
```

3.1 `x_t`

`x_t` 是当前 WAN diffusion step 的 noisy latent，也就是正在采样中的状态。它来自 scheduler 当前的 `self.latents`。

3.2 `x0_pred`

用当前 step 的噪声预测 `eps` 反推出近似干净 latent：

```
x0_pred = x_t - sigma_t * eps
```

这里仍然沿用 WAN scheduler 的语义。也就是说，超分不是直接作用在 noisy latent 上，而是先作用在近似 clean latent 上。

3.3 `WAN decode`

把低分 WAN clean latent 解码成低分 RGB 视频：

```
WAN latent -> RGB video
```

这样做的原因是：WAN latent 和 LTX2 latent 的通道数、统计分布、语义空间都不同，不能无训练地直接相互替换。

3.4 `LTX2 encode`

把低分 RGB 视频重新编码到 LTX2 video VAE latent 空间：

```
RGB video -> LTX2 latent
```

这一步把数据送进 LTX2 upsampler 原生认识的 latent 空间。

3.5 `LTX2 upsample`

调用 LTX2 spatial x2 latent upsampler：

```
low-res LTX2 latent -> high-res LTX2 latent
```

这是当前方案复用 LTX2 权重的核心步骤。

3.6 `LTX2 decode`

把高分 LTX2 latent 解码回高分 RGB 视频：

```
high-res LTX2 latent -> high-res RGB video
```

3.7 `WAN encode`

把高分 RGB 视频重新编码回 WAN latent：

```
high-res RGB video -> high-res WAN latent
```

这样后续才能回到 WAN diffusion 轨道继续采样。

3.8 `add_noise`

把高分 WAN clean latent 重新加噪成当前 step 可接续的 high-res noisy latent：

```
x_t_next = alpha_t * x0_highres + sigma_t * noise_highres
```

如果没有这一步，后续 WAN scheduler 无法正常继续 diffusion。

3.9 `continue diffusion`

把 scheduler 的 `self.latents` 替换成新的 high-res noisy latent，并重建 timestep / sigma 日程，然后后续 step 继续走 WAN 原始去噪逻辑。

4. 当前配置下的 shape

当前配置：

```
{
  "target_video_length": 81,
  "target_height": 512,
  "target_width": 832,
  "resolution_rate": [0.5],
```

```
  "changing_resolution_steps": [25]
}
```

WAN 默认 VAE stride 是：

```
vae_stride = [4, 8, 8]
```

因此 shape 路径大致是：

```
低分 WAN latent:
[16, 21, 32, 52]

WAN decode 后低分 RGB:
[1, 3, 81, 256, 416]

LTX2 encode 后低分 LTX2 latent:
[128, 11, 8, 13]

LTX2 upsample 后高分 LTX2 latent:
[1, 128, 11, 16, 26]

LTX2 decode 后高分 RGB:
[1, 3, 81, 512, 832]

WAN encode 后高分 WAN latent:
[16, 21, 64, 104]

add_noise 后高分 WAN noisy latent:
[16, 21, 64, 104]
```

选择 `512x832` 的原因：

- 最终高宽都能被 `64` 整除。
- 低分阶段 `256x416` 都能被 LTX2 video VAE 的空间压缩比例 `32` 整除。
- 避免在 bridge 内部额外 pad/crop。

5. 与原始 changing_resolution 的对照

阶段	原始 changing_resolution	当前 LTX2 bridge 链路
当前状态	`x_t` noisy WAN latent	同上
近似 clean latent	`x0_pred = x_t - sigma_t * eps`	同上
分辨率迁移	`trilinear interpolate(x0_pred)`	WAN decode -> LTX2 encode -> LTX2 upsample -> LTX2 decode -> WAN encode

阶段	原始 changing_resolution	当前 LTX2 bridge 链路
重新接回 diffusion	`add_noise`	同上
后续采样	WAN scheduler 继续去噪	同上
计算成本	低	高
理论细节恢复能力	插值级别	依赖 LTX2 upsampler 权重

6. 运行方式

脚本入口：

```
scripts/changing_resolution/run_wan_t2v_ltx2_bridge.sh
```

核心命令：

```
python -m lightx2v.infer \
  --model_cls wan2.1_ltx2_bridge \
  --task t2v \
  --model_path $model_path \
  --config_json
${lightx2v_path}/configs/changing_resolution/wan_t2v_ltx2_bridge.json \
  --prompt "..." \
  --negative_prompt "..." \
  --save_result_path
${lightx2v_path}/save_results/output_lightx2v_wan_t2v_ltx2_bridge.mp4
```

注意：当前配置里的 LTX2 权重路径仍需要指向本地真实 `.safetensors` 文件。`LTX2VideoVAE` / `LTX2Upsampler` 底层 loader 使用 `safetensors.safe_open(path, ...)`，不会自动把 `"Lightricks/LTX-2/..."` 当作 Hugging Face repo 下载。

需要准备：

```
"ltx2_vae_ckpt": "/path/to/ltx-2-19b-distilled-fp8.safetensors",
"ltx2_upsampler_ckpt": "/path/to/ltx-2-spatial-upscaler-x2-1.0.safetensors"
```

7. 已知风险

7.1 性能和显存成本高

切分辨率那一步会额外执行：

- WAN VAE decode

- LTX2 VAE encode
- LTX2 latent upsample
- LTX2 VAE decode
- WAN VAE encode

这比原始 `trilinear interpolate` 重很多。

7.2 LTX2 和 WAN 的 VAE 往返可能带来损失

这条方案不是纯 latent bridge，而是借道 RGB。优点是无训练可用，缺点是会经过两套 VAE 的编解码，可能引入：

- 颜色漂移
- 细节重构误差
- temporal consistency 损失
- VAE 压缩噪声

7.3 只适合 x2 升分

当前复用的是 LTX2 spatial x2 upsampler，因此配置必须匹配：

```
"resolution_rate": [0.5]
```

不建议直接改回 `0.75 -> 1.0`，因为那是 `4/3` 倍率，不匹配当前 LTX2 x2 权重。

7.4 帧数需要兼容 LTX2 VAE

LTX2 video VAE 要求输入帧数满足：

```
frames = 1 + 8 * k
```

当前 `81` 帧满足。如果后续改帧数，需要同步检查。

7.5 尺寸建议保持最终高宽能被 64 整除

最终尺寸如果不是 `64` 的倍数，则低分阶段可能不是 LTX2 `32` 网格的整数倍，需要额外 pad/crop。当前 `512x832` 是为了避免这个问题。

8. 后续优化方向

8.1 运行前强校验

建议在 `WanLTX2BridgeRunner` 中补充更严格的启动检查：

- `target_video_length - 1` 是否能被 `8` 整除。
- `target_height` / `target_width` 是否能被 `64` 整除。
- LTX2 ckpt 是否是本地存在的 `.safetensors` 文件。

- `changing_resolution_steps` 是否只包含一个 step。
- `resolution_rate` 是否严格等于 `[0.5]`。

8.2 本地 ckpt 路径解析

当前配置直接使用 `ltx2_vae_ckpt` 和 `ltx2_upsampler_ckpt`。后续可以增加类似 WAN 的 `find_torch_model_path` 逻辑，例如：

```
model_path/LTX-2/ltx-2-19b-distilled-fp8.safetensors
model_path/LTX-2/ltx-2-spatial-upscaler-x2-1.0.safetensors
```

这样脚本更容易复用。

8.3 切换 step 搜索

当前切换步是第 `25` 步。后续需要 A/B：

- 第 `15` 步切换
- 第 `20` 步切换
- 第 `25` 步切换
- 第 `30` 步切换

观察清晰度、稳定性、噪声残留、结构崩坏和时序一致性。

8.4 add_noise sigma 策略

当前仍沿用原 changing_resolution 的 `add_noise` 策略。后续可以比较：

- 使用当前 step 的 sigma。
- 使用下一 step 的 sigma。
- 切换后重设一段短 schedule。
- 类似 LTX2 stage2 的 refinement sigma 设计。

8.5 与原始插值链路做固定 seed 对比

建议固定：

- prompt
- negative prompt
- seed
- target size
- infer steps
- switching step

对比：

- 原始 `trilinear`
- 当前 `LTX2 bridge`

重点看：

- 细节是否更清晰
- 是否引入颜色漂移
- 是否有 VAE 往返导致的纹理污染
- 后续 WAN diffusion 是否能修复 bridge 引入的误差

9. 当前结论

当前方案是一条无训练条件下复用 LTX2 upsampler 权重的工程折中：

- 它比直接对 WAN latent 做插值更有潜力恢复高频细节。
- 它避免了无训练 latent adapter 的不可靠通道映射。
- 它代价较高，因为需要借道 RGB 和两套 VAE。
- 它目前应该作为实验链路，而不是默认替代原 `changing_resolution`。

一句话总结：

```
保留 WAN diffusion 调度，只把 x0_pred 的分辨率迁移阶段替换成 LTX2 原生 latent
upsample 能处理的 RGB/VAE 桥接流程。
```